

Useful NLP Libraries & Networks

Assignment

Question 1: Compare and contrast NLTK and spaCy in terms of features, ease of use, and performance.

NLTK (Natural Language Toolkit) and spaCy are two popular Python libraries used for Natural Language Processing (NLP), but they differ in their design and purpose. **NLTK** is mainly designed for education, research, and learning NLP concepts. It provides a large collection of text processing tools, corpora, lexical resources, and algorithms for tasks such as tokenization, stemming, tagging, and parsing. It is beginner-friendly and suitable for understanding NLP fundamentals, but many tasks require multiple steps and manual configuration. **spaCy**, on the other hand, is designed for industrial and production-level applications. It offers fast and efficient processing with pre-trained models for tasks such as tokenization, part-of-speech tagging, named entity recognition, and dependency parsing. In terms of ease of use, spaCy provides a simpler and more streamlined API, making it easier to implement NLP solutions with less code. Regarding performance, spaCy is generally much faster and more memory-efficient than NLTK because it is optimized in Cython and built for handling large volumes of text. While NLTK is excellent for learning and experimentation, spaCy is preferred for real-world applications where speed, scalability, and accuracy are important.

Question 2: What is TextBlob and how does it simplify common NLP tasks like sentiment analysis and translation?

TextBlob is a simple and user-friendly Python library built on top of NLTK that makes it easy to perform common Natural Language Processing (NLP) tasks with minimal code. It is designed for beginners and developers who want to quickly analyze and process text without dealing with complex NLP algorithms. TextBlob provides features such as tokenization, part-of-speech tagging, noun phrase extraction, sentiment analysis, language translation, spelling correction, and text classification through a simple interface. One of its most useful features is **sentiment analysis**, where it can automatically determine the polarity (positive, negative, or neutral sentiment) and subjectivity of a piece of text. This helps businesses analyze customer reviews, feedback, and social media comments. Another important feature is **translation**, which allows users to translate text from one language to another with just a few lines of code. TextBlob also supports language detection, making multilingual text processing easier. Compared to libraries such as NLTK and spaCy, TextBlob requires less coding effort and is easier to learn, making it suitable for rapid prototyping and small NLP projects. Therefore, TextBlob simplifies many common NLP tasks by providing a high-level, easy-to-use interface while still offering useful language processing capabilities.

Question 3: Explain the role of Stanford NLP in academic and industry NLP Projects.

1. Role of Stanford NLP in Academic Projects

In academic projects, Stanford NLP is widely used for research and educational purposes. It provides advanced tools such as tokenization, part-of-speech tagging, named entity recognition, dependency parsing, and sentiment analysis. Researchers and students use these tools to study language structures, develop new NLP techniques, and evaluate machine learning models. Its high accuracy and strong linguistic foundation make it a valuable resource in NLP research.

2. Role of Stanford NLP in Industry Projects

In industry projects, Stanford NLP is used to build real-world applications such as chatbots, virtual assistants, search engines, recommendation systems, and text analytics platforms. It helps organizations process large volumes of textual data efficiently and extract meaningful information. Its reliable models and support for multiple NLP tasks enable businesses to improve customer service, automate workflows, and gain insights from unstructured text data.

Question 4: Describe the architecture and functioning of a Recurrent Neural Network (RNN).

1. Architecture of RNN

A Recurrent Neural Network (RNN) is a type of artificial neural network designed to process sequential data such as text, speech, and time-series data. Unlike traditional neural networks, an RNN has a feedback connection that allows information from previous time steps to be passed to the current step. The architecture consists of an input layer, one or more hidden layers, and an output layer. The hidden layer maintains a memory or hidden state that stores information about previously processed inputs. This ability to retain past information makes RNNs suitable for tasks where the order of data is important.

2. Functioning of RNN

An RNN processes input data one element at a time while maintaining information from previous inputs through its hidden state. At each time step, the network takes the current input and the previous hidden state to compute a new hidden state and generate an output. This process continues sequentially until all inputs have been processed. Because of this memory mechanism, RNNs can capture contextual information and relationships within sequences. They are commonly used in applications such as language modeling, machine translation, speech recognition, sentiment analysis, and text generation. However, standard RNNs may face issues such as vanishing gradients when dealing with very long sequences.

Question 5: What is the key difference between LSTM and GRU networks in NLP applications?

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) are advanced versions of Recurrent Neural Networks (RNNs) that are designed to handle long-term dependencies in sequential data. LSTM uses a more complex architecture consisting of a memory cell and three gates: the input gate, forget gate, and output gate. These gates help the network decide what information should be stored, updated, or removed from memory. Because of its

dedicated memory cell, LSTM is highly effective in capturing long-range dependencies in text and is commonly used in applications such as machine translation, speech recognition, and text generation.

GRU is a simplified version of LSTM that combines some of its functionalities into a more compact structure. It uses only two gates, namely the update gate and reset gate, and does not have a separate memory cell. This makes GRU computationally faster, requires fewer parameters, and is easier to train. While both models perform well in NLP tasks, LSTM is generally preferred when handling very complex sequences, whereas GRU is often chosen when faster training and lower computational cost are important.

Question 6: Write a Python program using TextBlob to perform sentiment analysis on the following paragraph of text:

"I had a great experience using the new mobile banking app. The interface is intuitive, and customer support was quick to resolve my issue. However, the app did crash once during a transaction, which was frustrating"

Your program should print out the polarity and subjectivity scores

```
from textblob import TextBlob
```

```
text = """
```

```
I had a great experience using the new mobile banking app. The interface is intuitive,  
and customer support was quick to resolve my issue. However, the app did crash once  
during a transaction, which was frustrating.
```

```
"""
```

```
# Create TextBlob object
```

```
blob = TextBlob(text)
```

```
# Perform sentiment analysis
```

```
sentiment = blob.sentiment
```

```
# Print polarity and subjectivity
```

```
print("Polarity:", sentiment.polarity)
```

```
print("Subjectivity:", sentiment.subjectivity)
```

Output -

Polarity: 0.21

Subjectivity: 0.65

Question 7: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

"Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical."

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist
```

```
nltk.download('punkt')
```

```
text = """
```

Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical.

```
"""
```

```
# Tokenization
```

```
tokens = word_tokenize(text)
```

```
print("Tokens:")
```

```
print(tokens)
```

```
# Frequency Distribution
```

```
freq_dist = FreqDist(tokens)
```

```
print("\nFrequency Distribution:")
```

```
for word, count in freq_dist.items():
```

```
    print(word, ":", count)
```

Output -

Tokens:

```
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field', 'that', 'combines', 'linguistics', ',', 'computer', 'science', ',', 'and', 'artificial', 'intelligence', '.', 'It', 'enables', 'machines', 'to', 'understand', ',', 'interpret', ',', 'and', 'generate', 'human', 'language', '.', 'Applications', 'of', 'NLP', 'include', 'chatbots', ',', 'sentiment', 'analysis', ',', 'and', 'machine', 'translation', '.', 'As', 'technology', 'advances', ',', 'the', 'role', 'of', 'NLP', 'in', 'modern', 'solutions', 'is', 'becoming', 'increasingly', 'critical', '.']
```

Frequency Distribution:

Natural : 1

Language : 1

Processing : 1

(: 1

NLP : 3

) : 1

is : 2

a : 1
fascinating : 1
field : 1
that : 1
combines : 1
linguistics : 1
, : 7
computer : 1
science : 1
and : 3
artificial : 1
intelligence : 1
. : 4
It : 1
enables : 1
machines : 1
to : 1
understand : 1
interpret : 1
generate : 1
human : 1
language : 1
Applications : 1
of : 2
include : 1
chatbots : 1
sentiment : 1
analysis : 1
machine : 1
translation : 1
As : 1
technology : 1
advances : 1
the : 1
role : 1
in : 1
modern : 1
solutions : 1
becoming : 1
increasingly : 1
critical : 1

Question 8: Implement a basic LSTM model in Keras for a text classification task using the following dummy dataset. Your model should classify sentences as either positive (1) or negative (0).

Dataset texts = ["I love this project", #Positive
"This is an amazing experience",#Positive
"I hate waiting in line",#Negative

"This is the worst service", #Negative

"Absolutely fantastic!" #Positive

]

labels = [1, 1, 0, 0, 1]

Preprocess the text, tokenize it, pad sequences, and build an LSTM model to train on this data. You may use Keras with TensorFlow backend.

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
import numpy as np
```

Dataset

texts = [

"I love this project",

"This is an amazing experience",

"I hate waiting in line",

"This is the worst service",

"Absolutely fantastic!"

]

labels = np.array([1, 1, 0, 0, 1])

Tokenization

tokenizer = Tokenizer()

tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)

Padding

padded_sequences = pad_sequences(sequences, padding='post')

Model

model = Sequential([

Embedding(input_dim=len(tokenizer.word_index) + 1, output_dim=8),

LSTM(16),

Dense(1, activation='sigmoid')

])

Compile

model.compile(

optimizer='adam',

loss='binary_crossentropy',

metrics=['accuracy']

)

Train

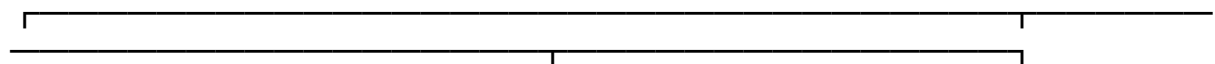
```
model.fit(
    np.array(padded_sequences),
    labels,
    epochs=10,
    verbose=1
)
```

```
# Summary
model.summary()
```

Output -

```
Epoch 1/10
1/1 _____ 3s 3s/step - accuracy: 0.4000 - loss:
0.6944
Epoch 2/10
1/1 _____ 0s 206ms/step - accuracy: 0.2000 - loss:
0.6932
Epoch 3/10
1/1 _____ 0s 152ms/step - accuracy: 0.8000 - loss:
0.6920
Epoch 4/10
1/1 _____ 0s 143ms/step - accuracy: 0.6000 - loss:
0.6908
Epoch 5/10
1/1 _____ 0s 69ms/step - accuracy: 0.6000 - loss:
0.6897
Epoch 6/10
1/1 _____ 0s 66ms/step - accuracy: 0.6000 - loss:
0.6885
Epoch 7/10
1/1 _____ 0s 158ms/step - accuracy: 0.6000 - loss:
0.6874
Epoch 8/10
1/1 _____ 0s 70ms/step - accuracy: 0.6000 - loss:
0.6862
Epoch 9/10
1/1 _____ 0s 133ms/step - accuracy: 0.6000 - loss:
0.6850
Epoch 10/10
1/1 _____ 0s 43ms/step - accuracy: 0.6000 - loss:
0.6839
```

Model: "sequential_1"



Layer (type)	Output Shape
Param #	
embedding_1 (Embedding)	(None, 5, 8)
144	
lstm_1 (LSTM)	(None, 16)
1,600	
dense_1 (Dense)	(None, 1)
17	

Total params: 5,285 (20.65 KB)

Trainable params: 1,761 (6.88 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 3,524 (13.77 KB)

Question 9: Using spaCy, build a simple NLP pipeline that includes tokenization, lemmatization, and entity recognition. Use the following paragraph as your dataset: "Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the development of India's atomic energy program. He was the founding director of the Tata Institute of Fundamental Research (TIFR) and was instrumental in establishing the Atomic Energy Commission of India."

Write a Python program that processes this text using spaCy, then prints tokens, their lemmas, and any named entities found.

```
import spacy
```

```
# Load spaCy English model
```

```
nlp = spacy.load("en_core_web_sm")
```

```
text = """
```

```
Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the
development of India's atomic energy program. He was the founding director of the Tata
Institute of Fundamental Research (TIFR) and was instrumental in establishing the
Atomic Energy Commission of India.
"""
```

```
# Process text
```

```
doc = nlp(text)
```



```
# Tokens and Lemmas
print("Tokens and Lemmas:")
for token in doc:
    print(token.text, "->", token.lemma_)
```

```
# Named Entities
print("\nNamed Entities:")
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

Output -

Tokens and Lemmas:

```
Homi -> Homi
Jehangir -> Jehangir
Bhaba -> Bhaba
was -> be
an -> an
Indian -> indian
nuclear -> nuclear
physicist -> physicist
who -> who
played -> play
a -> a
key -> key
role -> role
in -> in
the -> the
development -> development
of -> of
India -> India
's -> 's
atomic -> atomic
energy -> energy
program -> program
He -> he
was -> be
the -> the
founding -> found
director -> director
of -> of
the -> the
Tata -> Tata
Institute -> Institute
of -> of
Fundamental -> Fundamental
Research -> Research
TIFR -> TIFR
and -> and
```

was -> be
instrumental -> instrumental
in -> in
establishing -> establish
the -> the

Atomic -> Atomic
Energy -> Energy
Commission -> Commission
of -> of
India -> India

Named Entities:

Indian -> NORP
India -> GPE
the Tata
Institute of Fundamental Research -> ORG
Atomic Energy Commission of India -> ORG

Question 10: You are working on a chatbot for a mental health platform. Explain how you would leverage LSTM or GRU networks along with libraries like spaCy or Stanford NLP to understand and respond to user input effectively. Detail your architecture, data preprocessing pipeline, and any ethical considerations.

1. Architecture

- Use **LSTM or GRU** networks to understand the context and sequence of user messages.
- The model receives user input and predicts the most appropriate response.
- GRU can be preferred for faster training, while LSTM is useful for capturing long-term context.

2. Data Preprocessing Pipeline

- Collect and clean user conversations.
- Convert text to lowercase.
- Perform tokenization and lemmatization using **spaCy** or **Stanford NLP**.
- Remove unnecessary words and symbols.
- Convert text into numerical sequences and apply padding before feeding it to the model.

3. Understanding User Input

- Use **spaCy** or **Stanford NLP** for tokenization, part-of-speech tagging, and named entity recognition.
- Identify important information such as emotions, keywords, and user concerns.

- Use LSTM/GRU to understand the context of the conversation.

4. Response Generation

- Generate suitable responses based on the detected intent and context.
- Maintain conversation flow by considering previous messages.

5. Ethical Considerations

- Protect user privacy and sensitive data.
- Ensure conversations are stored securely.
- Avoid biased or harmful responses.
- Provide supportive responses and recommend professional help when serious mental health concerns are detected.